



Deep Learning

Tutorial 16: Generative Adversarial Network (GANs)

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

Change the number of epochs, layers etc
Replace fully connected layers with **convolutional layers** for image generation tasks.
Develop the model for augmented images of any of the study you conducted during the entire course.

Implementing the Task in Pytorch:

```
# =====
```

```
# Tutorial 16 – Generative Adversarial Networks (GANs)
```

```
# Implemented in PyTorch | Ready for Google Colab
```

```
# =====
```

```
# Objectives:
```

```
# • Understand the architecture of GANs
```

```
# • How to implement GANs in PyTorch
```

```
# • Visualize generated results
```

```
#
```

```
# TASKS COVERED:
```

```
# ✓ Vanilla GAN (fully-connected) on MNIST
```

```
# ✓ Change epochs, layers, and hyperparameters
```

```
# ✓ DCGAN (Deep Convolutional GAN) – convolutional layers
```

```
# ✓ GAN for data augmentation (Fashion-MNIST as study example)
```

```
# =====
```

```
# — 0. Imports —————
```

```

import torch

import torch.nn as nn

import torch.optim as optim

from torch.utils.data import DataLoader

from torchvision import datasets, transforms

import matplotlib.pyplot as plt

import numpy as np


torch.manual_seed(42)

np.random.seed(42)


device = "cuda" if torch.cuda.is_available() else "cpu"

print(f"Using device: {device}")


# =====

# SECTION 1 – VANILLA GAN (Fully Connected) on MNIST

# =====

print("\n" + "="*60)

print("SECTION 1 – Vanilla GAN on MNIST")

print("="*60)


# — 1.1 Load & prepare MNIST —————

BATCH_SIZE = 64

```

```
IMAGE_SIZE = 28 * 28    # flattened
```

```
LATENT_DIM = 100
```

```
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,)), # normalise to [-1, 1]
])
```

```
mnist_dataset = datasets.MNIST(root="./data", train=True,
                                transform=transform, download=True)
```

```
train_loader = DataLoader(mnist_dataset, batch_size=BATCH_SIZE, shuffle=True)
```

```
print(f"MNIST dataset loaded: {len(mnist_dataset)} images")
```

```
# — 1.2 Generator —————
```

```
class Generator(nn.Module):
```

```
    """
```

```
    Latent vector → fully-connected layers → 28×28 image
```

```
    """
```

```
    def __init__(self, latent_dim=100):
```

```
        super().__init__()
```

```
        self.model = nn.Sequential(
```

```
            nn.Linear(latent_dim, 128),
```

```
            nn.ReLU(),
```

```

nn.Linear(128, 256),

nn.ReLU(),

nn.Linear(256, 512),

nn.ReLU(),

nn.Linear(512, 784),

nn.Tanh()          # output in [-1, 1]

)

```

```

def forward(self, z):

    return self.model(z).view(z.size(0), 1, 28, 28)

```

— 1.3 Discriminator —————

```

class Discriminator(nn.Module):

    """

    28×28 image → flatten → FC layers → probability (real/fake)

    """

    def __init__(self):

        super().__init__()

        self.model = nn.Sequential(

            nn.Linear(784, 512),

            nn.LeakyReLU(0.2),

            nn.Linear(512, 256),

            nn.LeakyReLU(0.2),

```

```
nn.Linear(256, 1),  
nn.Sigmoid()  
)
```

```
def forward(self, x):
```

```
    x = x.view(x.size(0), -1)    # flatten
```

```
    return self.model(x)
```

```
# — 1.4 Loss & optimisers —————
```

```
criterion = nn.BCELoss()
```

```
lr      = 0.0002
```

```
generator  = Generator(LATENT_DIM).to(device)
```

```
discriminator = Discriminator().to(device)
```

```
gen_optimizer = optim.Adam(generator.parameters(), lr=lr, betas=(0.5, 0.999))
```

```
disc_optimizer = optim.Adam(discriminator.parameters(), lr=lr, betas=(0.5, 0.999))
```

```
# — 1.5 Visualisation helper —————
```

```
def visualize_images(generator, latent_dim, num_images=16,
```

```
    title="Generated Images", save_path=None):
```

```
    generator.eval()
```

```
    with torch.no_grad():
```

```

noise = torch.randn(num_images, latent_dim).to(device)

images = generator(noise).detach().cpu()


fig, axes = plt.subplots(1, num_images, figsize=(num_images, 1.5))

for i in range(num_images):

    axes[i].imshow(images[i].squeeze(), cmap="gray",

                    vmin=-1, vmax=1)

    axes[i].axis("off")

plt.suptitle(title, y=1.02)

plt.tight_layout()

if save_path:

    plt.savefig(save_path, dpi=150, bbox_inches="tight")

plt.show()

generator.train()


# — 1.6 GAN training function —————

def train_gan(generator, discriminator, train_loader,

               gen_optimizer, disc_optimizer, criterion,

               epochs, latent_dim, visualise_every=10,

               model_name="VanillaGAN"):

    g_losses, d_losses = [], []

```

```
for epoch in range(epochs):
```

```
    g_epoch, d_epoch, batches = 0, 0, 0
```

```
    for real_images, _ in train_loader:
```

```
        real_images = real_images.to(device)
```

```
        batch_size = real_images.size(0)
```

```
        real_labels = torch.ones(batch_size, 1).to(device)
```

```
        fake_labels = torch.zeros(batch_size, 1).to(device)
```

```
        # — Train Discriminator —————
```

```
        disc_optimizer.zero_grad()
```

```
        real_outputs = discriminator(real_images)
```

```
        real_loss = criterion(real_outputs, real_labels)
```

```
        noise = torch.randn(batch_size, latent_dim).to(device)
```

```
        fake_images = generator(noise)
```

```
        fake_outputs = discriminator(fake_images.detach())
```

```
        fake_loss = criterion(fake_outputs, fake_labels)
```

```
        d_loss = real_loss + fake_loss
```

```
        d_loss.backward()
```



```

disc_optimizer.step()

# — Train Generator —————

gen_optimizer.zero_grad()

noise      = torch.randn(batch_size, latent_dim).to(device)
fake_images = generator(noise)
fake_outputs = discriminator(fake_images)
g_loss      = criterion(fake_outputs, real_labels) # fool D
g_loss.backward()
gen_optimizer.step()

g_epoch += g_loss.item()
d_epoch += d_loss.item()
batches += 1

g_losses.append(g_epoch / batches)
d_losses.append(d_epoch / batches)

print(f" [{model_name}] Epoch [{epoch+1}/{epochs}] "
      f"D Loss: {d_losses[-1]:.4f} | G Loss: {g_losses[-1]:.4f}")

if (epoch + 1) % visualise_every == 0:

```

```

        visualize_images(generator, latent_dim,

                        title=f"{model_name} – Epoch {epoch+1}",

                        save_path=f"{model_name}_epoch{epoch+1}.png")

    return g_losses, d_losses

# — 1.7 Train Vanilla GAN —————

# Task: Change number of epochs, layers etc.

# Original: 50 epochs. We run 20 here for speed; change freely.

EPOCHS_VANILLA = 20

print(f"\nTraining Vanilla GAN for {EPOCHS_VANILLA} epochs ...")

g_loss_v, d_loss_v = train_gan(

    generator, discriminator, train_loader,

    gen_optimizer, disc_optimizer, criterion,

    epochs      = EPOCHS_VANILLA,

    latent_dim   = LATENT_DIM,

    visualise_every= 10,

    model_name   = "VanillaGAN"

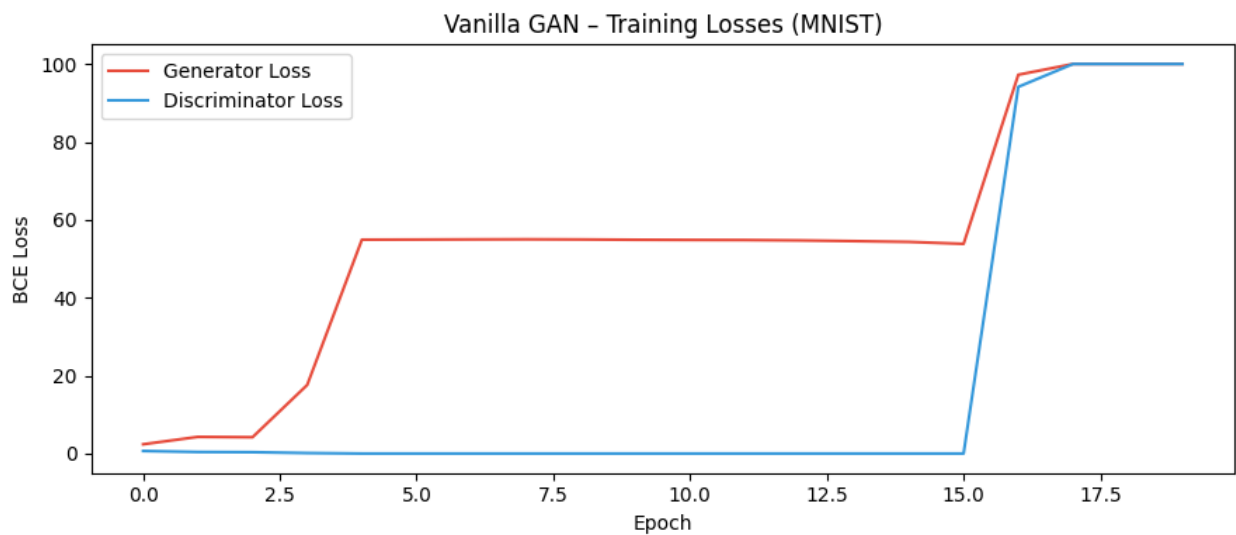
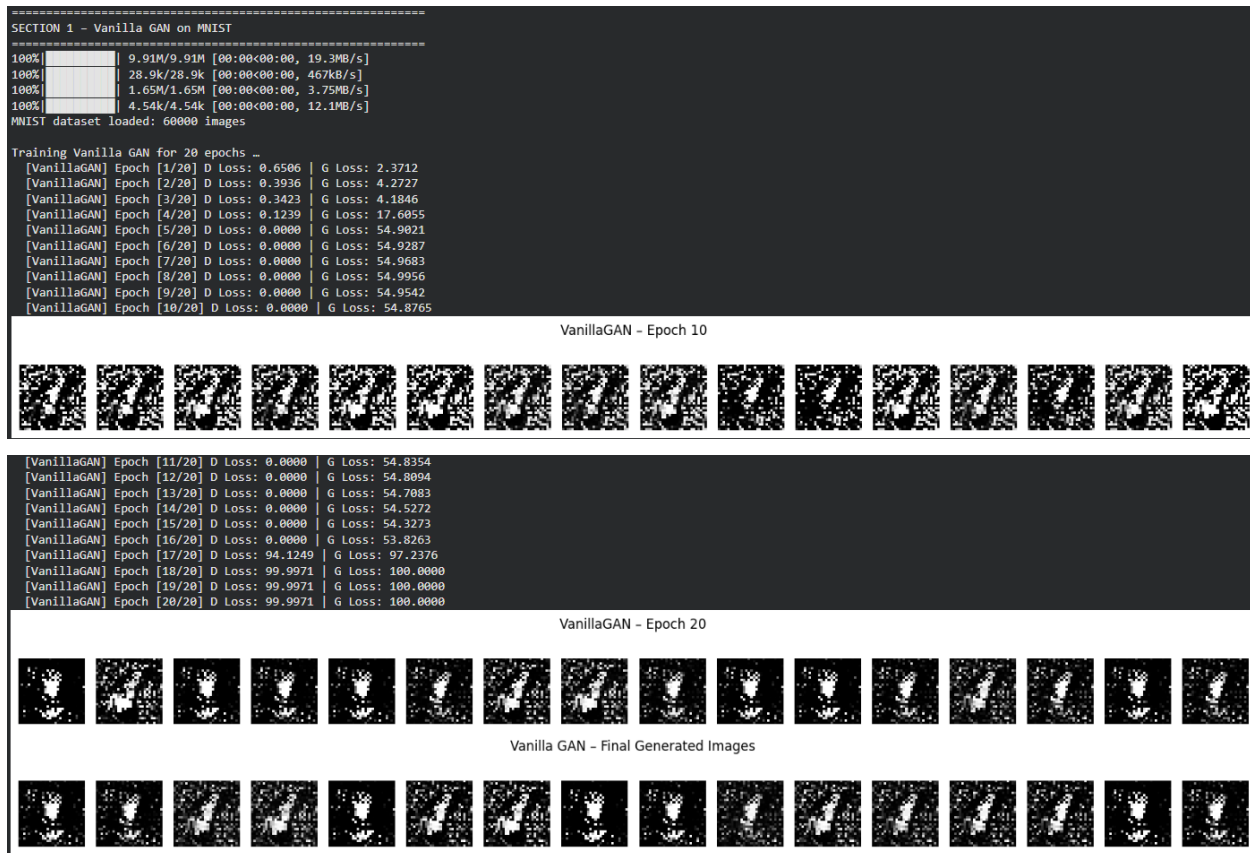
)

# Final visualization

visualize_images(generator, LATENT_DIM,

```

```
title="Vanilla GAN – Final Generated Images",  
save_path="Tutorial16_vanilla_gan_final.png")  
  
print("Saved: Tutorial16_vanilla_gan_final.png")  
  
# Loss curves  
  
plt.figure(figsize=(9, 4))  
  
plt.plot(g_loss_v, label="Generator Loss", color="#E74C3C")  
plt.plot(d_loss_v, label="Discriminator Loss", color="#3498DB")  
  
plt.title("Vanilla GAN – Training Losses (MNIST)")  
  
plt.xlabel("Epoch"); plt.ylabel("BCE Loss")  
  
plt.legend(); plt.tight_layout()  
  
plt.savefig("Tutorial16_vanilla_gan_losses.png", dpi=150)  
  
plt.show()  
  
print("Saved: Tutorial16_vanilla_gan_losses.png")
```



```

# =====

# SECTION 2 – DCGAN (Task: Replace FC with Convolutional Layers)

# =====

print("\n" + "="*60)

print("SECTION 2 – DCGAN (Convolutional GAN) on MNIST")

print("="*60)


LATENT_DIM_DC = 100


# — 2.1 DCGAN Generator (Transpose Convolutions) —————

class DCGenerator(nn.Module):

    """
    z (100) → ConvTranspose2d stack → 1×28×28
    """

    def __init__(self, latent_dim=100, ngf=64):

        super().__init__()

        self.net = nn.Sequential(

            # z: (B, latent_dim, 1, 1)

            nn.ConvTranspose2d(latent_dim, ngf * 4, 4, 1, 0, bias=False),

            nn.BatchNorm2d(ngf * 4),

            nn.ReLU(True),          # → 4×4

            nn.ConvTranspose2d(ngf * 4, ngf * 2, 3, 2, 1, bias=False),

```

```

nn.BatchNorm2d(ngf * 2),

nn.ReLU(True),          #  $\rightarrow 7 \times 7$ 

nn.ConvTranspose2d(ngf * 2, ngf, 4, 2, 1, bias=False),

nn.BatchNorm2d(ngf),

nn.ReLU(True),          #  $\rightarrow 14 \times 14$ 

nn.ConvTranspose2d(ngf, 1, 4, 2, 1, bias=False),

nn.Tanh()                #  $\rightarrow 28 \times 28$ , range  $[-1, 1]$ 

)

```

```
def forward(self, z):
```

```
    z = z.view(z.size(0), z.size(1), 1, 1)
```

```
    return self.net(z)
```

— 2.2 DCGAN Discriminator (Convolutions) —————

```
class DCDiscriminator(nn.Module):
```

```
    """
```

```
    1  $\times$  28  $\times$  28  $\rightarrow$  Conv2d stack  $\rightarrow$  probability
```

```
    """
```

```
def __init__(self, ndf=64):
```

```
    super().__init__()
```

```
    self.net = nn.Sequential(
```

```
nn.Conv2d(1, ndf, 4, 2, 1, bias=False),  
nn.LeakyReLU(0.2, inplace=True),    #  $\rightarrow 14 \times 14$ 
```

```
nn.Conv2d(ndf, ndf * 2, 4, 2, 1, bias=False),  
nn.BatchNorm2d(ndf * 2),  
nn.LeakyReLU(0.2, inplace=True),    #  $\rightarrow 7 \times 7$ 
```

```
nn.Conv2d(ndf * 2, ndf * 4, 3, 2, 1, bias=False),  
nn.BatchNorm2d(ndf * 4),  
nn.LeakyReLU(0.2, inplace=True),    #  $\rightarrow 4 \times 4$ 
```

```
nn.Conv2d(ndf * 4, 1, 4, 1, 0, bias=False),  
nn.Sigmoid()                        #  $\rightarrow 1 \times 1 \times 1$ 
```

```
)
```

```
def forward(self, x):
```

```
    return self.net(x).view(-1, 1)
```

```
def weights_init(m):
```

```
    """DCGAN weight initialisation from the original paper."""
```

```
    classname = m.__class__.__name__
```

```
    if classname.find("Conv") != -1:
```

```
        nn.init.normal_(m.weight.data, 0.0, 0.02)
```

```

elif classname.find("BatchNorm") != -1:

    nn.init.normal_(m.weight.data, 1.0, 0.02)

    nn.init.constant_(m.bias.data, 0)


dc_gen = DCGenerator(LATENT_DIM_DC).to(device)

dc_disc = DCDiscriminator().to(device)

dc_gen.apply(weights_init)

dc_disc.apply(weights_init)


dc_criterion = nn.BCELoss()

dc_gen_opt = optim.Adam(dc_gen.parameters(), lr=0.0002, betas=(0.5, 0.999))

dc_disc_opt = optim.Adam(dc_disc.parameters(), lr=0.0002, betas=(0.5, 0.999))


EPOCHS_DC = 15

print(f"\nTraining DCGAN for {EPOCHS_DC} epochs ...")

g_loss_dc, d_loss_dc = train_gan(

    dc_gen, dc_disc, train_loader,

    dc_gen_opt, dc_disc_opt, dc_criterion,

    epochs = EPOCHS_DC,

    latent_dim = LATENT_DIM_DC,

    visualise_every= 5,

    model_name = "DCGAN"

)

```



```

visualize_images(dc_gen, LATENT_DIM_DC,
                 title="DCGAN – Final Generated Images",
                 save_path="Tutorial16_dcgan_final.png")

print("Saved: Tutorial16_dcgan_final.png")

```

```

# Loss curves

plt.figure(figsize=(9, 4))

plt.plot(g_loss_dc, label="DCGAN Generator", color="#E74C3C")
plt.plot(d_loss_dc, label="DCGAN Discriminator", color="#3498DB")

plt.title("DCGAN – Training Losses (MNIST)")

plt.xlabel("Epoch"); plt.ylabel("BCE Loss")

plt.legend(); plt.tight_layout()

plt.savefig("Tutorial16_dcgan_losses.png", dpi=150)

plt.show()

print("Saved: Tutorial16_dcgan_losses.png")

```

```

# — 2.3 Vanilla vs DCGAN loss comparison —————

min_len = min(len(g_loss_v), len(g_loss_dc))

plt.figure(figsize=(10, 4))

plt.plot(g_loss_v[:min_len], label="Vanilla GAN – G", color="#E74C3C", linestyle="-")
plt.plot(g_loss_dc[:min_len], label="DCGAN – G", color="#E67E22", linestyle="--")
plt.plot(d_loss_v[:min_len], label="Vanilla GAN – D", color="#3498DB", linestyle="-")

```

```
plt.plot(d_loss_dc[:min_len], label="DCGAN - D", color="#2980B9", linestyle="--")

plt.title("Vanilla GAN vs DCGAN – Generator & Discriminator Loss")

plt.xlabel("Epoch"); plt.ylabel("BCE Loss")

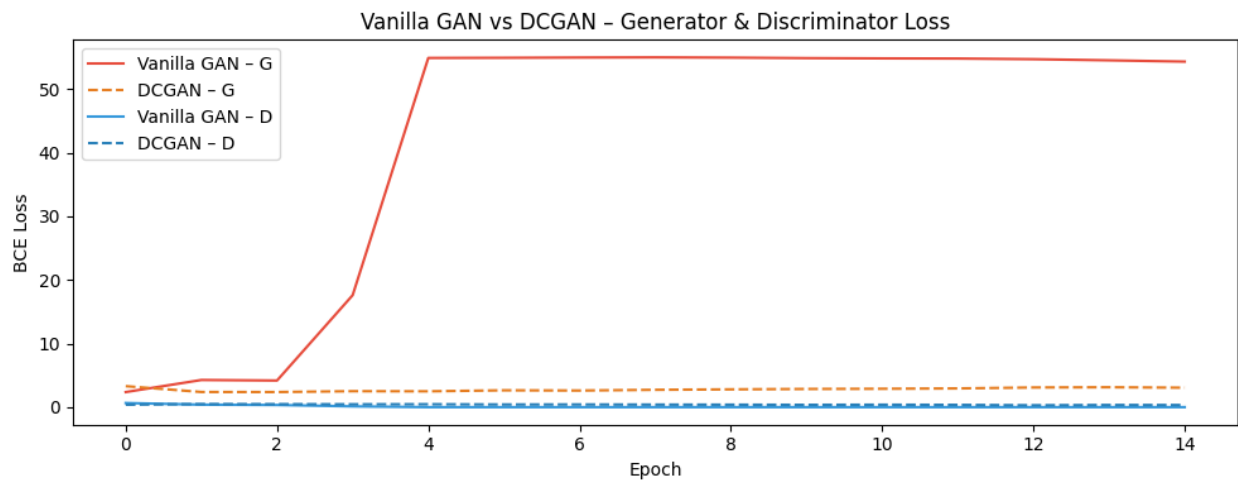
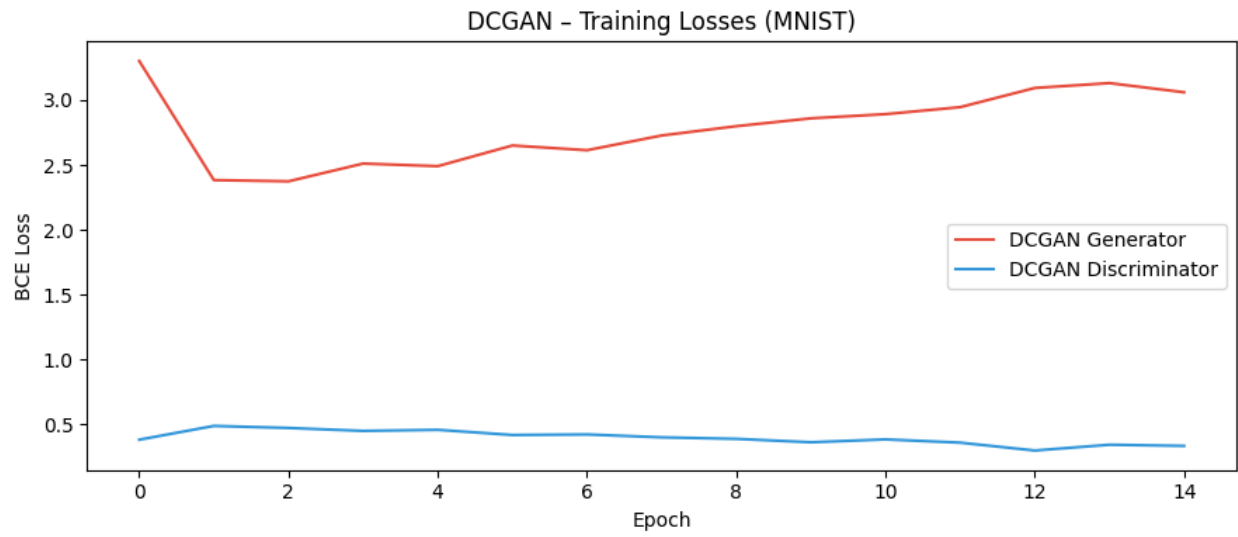
plt.legend(); plt.tight_layout()

plt.savefig("Tutorial16_comparison.png", dpi=150)

plt.show()

print("Saved: Tutorial16_comparison.png")
```





```
# =====
```

```
# SECTION 3 – GAN for Data Augmentation (Task 3)
```

```
# Study: Fashion-MNIST (garment recognition dataset)
```

```
# We train a small DCGAN and generate augmented samples.
```

```
# =====
```

```
print("\n" + "="*60)
```

```
print("SECTION 3 – GAN for Augmented Images (Fashion-MNIST)")
```

```
print("="*60)
```

```

fashion_dataset = datasets.FashionMNIST(root="./data", train=True,
                                         transform=transform, download=True)

fashion_loader = DataLoader(fashion_dataset, batch_size=BATCH_SIZE, shuffle=True)

CLASS_NAMES = ["T-shirt/top", "Trouser", "Pullover", "Dress", "Coat",
               "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]

# Use the same DCGAN architecture; re-initialise weights
aug_gen = DCGenerator(LATENT_DIM_DC).to(device)
aug_disc = DCDiscriminator().to(device)
aug_gen.apply(weights_init)
aug_disc.apply(weights_init)

aug_gen_opt = optim.Adam(aug_gen.parameters(), lr=0.0002, betas=(0.5, 0.999))
aug_disc_opt = optim.Adam(aug_disc.parameters(), lr=0.0002, betas=(0.5, 0.999))

EPOCHS_AUG = 10

print(f"\nTraining augmentation GAN (Fashion-MNIST) for {EPOCHS_AUG} epochs ...")

g_loss_aug, d_loss_aug = train_gan(
    aug_gen, aug_disc, fashion_loader,
    aug_gen_opt, aug_disc_opt, dc_criterion,
    epochs = EPOCHS_AUG,

```

```

latent_dim    = LATENT_DIM_DC,

visualise_every= 5,

model_name    = "AugGAN_FashionMNIST"
)

# Show real vs generated side by side

real_batch, real_cls = next(iter(fashion_loader))

real_batch = real_batch[:8]

aug_gen.eval()

with torch.no_grad():

    noise      = torch.randn(8, LATENT_DIM_DC).to(device)

    fake_images = aug_gen(noise).detach().cpu()

fig, axes = plt.subplots(2, 8, figsize=(16, 4))

for i in range(8):

    axes[0, i].imshow(real_batch[i].squeeze(), cmap="gray", vmin=-1, vmax=1)

    axes[0, i].set_title(CLASS_NAMES[real_cls[i].item()], fontsize=7)

    axes[0, i].axis("off")

    axes[1, i].imshow(fake_images[i].squeeze(), cmap="gray", vmin=-1, vmax=1)

    axes[1, i].set_title("Generated", fontsize=7)

    axes[1, i].axis("off")

```

```

axes[0, 0].set_ylabel("Real",    fontsize=10)

axes[1, 0].set_ylabel("Generated", fontsize=10)

plt.suptitle("Tutorial 16 – GAN Augmentation: Real vs Generated Fashion-MNIST")

plt.tight_layout()

plt.savefig("Tutorial16_fashion_augmentation.png", dpi=150, bbox_inches="tight")

plt.show()

print("Saved: Tutorial16_fashion_augmentation.png")

```

— Final summary —————

```
print("\n— Summary of Saved Outputs —")
```

```

outputs = [

    "Tutorial16_vanilla_gan_final.png",

    "Tutorial16_vanilla_gan_losses.png",

    "Tutorial16_dcgan_final.png",

    "Tutorial16_dcgan_losses.png",

    "Tutorial16_comparison.png",

    "Tutorial16_fashion_augmentation.png",

]

```

```
for f in outputs:
```

```
    print(f" • {f}")
```

```
print("\n✅ Tutorial 16 Complete!")
```

```

=====
SECTION 3 - GAN for Augmented Images (Fashion-MNIST)
=====
100% | 26.4M/26.4M [00:02:00:00, 12.8MB/s]
100% | 20.5k/20.5k [00:00:00:00, 201kB/s]
100% | 4.42M/4.42M [00:01:00:00, 3.74MB/s]
100% | 5.15k/5.15k [00:00:00:00, 25.3MB/s]

Training augmentation GAN (Fashion-MNIST) for 10 epochs ...
[AugGAN_FashionMNIST] Epoch [1/10] D Loss: 0.3807 | G Loss: 3.0724
[AugGAN_FashionMNIST] Epoch [2/10] D Loss: 0.6031 | G Loss: 2.2504
[AugGAN_FashionMNIST] Epoch [3/10] D Loss: 0.6521 | G Loss: 2.0518
[AugGAN_FashionMNIST] Epoch [4/10] D Loss: 0.6632 | G Loss: 1.9585
[AugGAN_FashionMNIST] Epoch [5/10] D Loss: 0.6408 | G Loss: 1.9907

AugGAN_FashionMNIST - Epoch 5

[AugGAN_FashionMNIST] Epoch [6/10] D Loss: 0.6283 | G Loss: 2.0308
[AugGAN_FashionMNIST] Epoch [7/10] D Loss: 0.6082 | G Loss: 2.0655
[AugGAN_FashionMNIST] Epoch [8/10] D Loss: 0.6131 | G Loss: 2.0954
[AugGAN_FashionMNIST] Epoch [9/10] D Loss: 0.5824 | G Loss: 2.1546
[AugGAN_FashionMNIST] Epoch [10/10] D Loss: 0.5836 | G Loss: 2.1771

AugGAN_FashionMNIST - Epoch 10

Tutorial 16 - GAN Augmentation: Real vs Generated Fashion-MNIST

Sandal Bag Dress Sandal Shirt Sandal Sneaker Shirt
Generated Generated Generated Generated Generated Generated Generated Generated
Generated Generated Generated Generated Generated Generated Generated Generated

— Summary of Saved Outputs —
• Tutorial16_vanilla_gan_final.png
• Tutorial16_vanilla_gan_losses.png
• Tutorial16_dcgan_final.png
• Tutorial16_dcgan_losses.png
• Tutorial16_comparison.png
• Tutorial16_fashion_augmentation.png

✅ Tutorial 16 Complete!

```